



**Faculty of Computers &
Artificial Intelligence**



Benha University

Disaster Tweet Classifier

➡ **Artificial Intelligence Department** ⬅

Presented by :

Ahmed Ayman Ahmed Thabet

Haidy Talaat Fadel Elseadawy

Mariam Mohamed Fathy Aboalata

Under Supervision of :

Eng. Mariam Mahmoud

Table of Contents

Chapter1	
1. Problem Definition.....	
1.1. Introduction.	
1.2. Problem Definition.	
1.3. Objectives.	
Chapter2	
2. Algorithm Selection and Implementation.....	
2.1. Dataset and Preprocessing.	
2.2. Model Selection.	
2.3. Training Implementation & Hyperparameters.	
Chapter3	
3. Optimization and Evaluation.....	
3.1 Hyperparameters optimization.....	
3.2 Optimization & Experimental Enhancements.....	
3.3 Evaluation Summary.....	
3.4 Model Evaluation.....	
Conclusion.....	

Chapter 1

Problem Definition and Environment Setup.

Chapter1

1.Problem Definition and Environment Setup.

1.1 Introduction.

In today's hyper-connected world, social media platforms like Twitter serve as critical channels for real-time information dissemination during emergencies. Millions of users share on-the-ground updates ranging from eyewitness accounts of natural disasters to requests for help and coordination among relief efforts. However, the sheer volume of posts makes manual monitoring infeasible for emergency response teams who need to quickly triage and respond to genuine crisis reports.

This project addresses this challenge by implementing a robust, scalable text classification pipeline that can automatically filter disaster-related tweets from general chatter. We leverage the power of transformer-based language models—specifically, a fine-tuned RoBERTa architecture—to capture nuanced linguistic patterns, contextual cues, and domain-specific terminology inherent in crisis communication. Our approach involves data preprocessing (cleaning, tokenization), model fine-tuning with dynamic padding and learning rate scheduling, and comprehensive evaluation using internal validation splits and external test data.

By integrating such an automated system, humanitarian organizations, first responders, and government agencies can benefit from:

- **Rapid insights:** Immediate detection of critical incident reports as they unfold.
- **Resource optimization:** Prioritization of high-impact messages reduces cognitive load and accelerates decision-making.

-
- **Scalability:** Ability to process thousands of tweets per second without manual intervention.

This solution aims not only for high predictive accuracy but also for generalizability across different types of disasters and geographic regions, ultimately contributing to more effective crisis management.

1.2 Problem Statement.

The task is a binary classification problem where each tweet must be labeled as either:

- **Disaster (1):** The tweet contains information about an actual disaster event or an emergency situation.
- **Non-Disaster (0):** The tweet does not relate to any real-world disaster and may be general conversation, personal updates, or unrelated content.

Given an input tweet text, the model learns to predict the correct label based on contextual cues, keywords, and semantic features extracted by the transformer encoder. The ultimate goal is to maximize accuracy and generalization ability, ensuring reliable performance on new, unseen tweets.

1.3 Objectives.

- **High Classification Accuracy:** Fine-tune the RoBERTa model to achieve at least 85% accuracy on unseen test data.
- **Robust Generalization:** Ensure the model performs well across various disaster types (natural, man-made) and geographic contexts.
- **Efficient Inference:** Optimize the pipeline for low-latency predictions to handle real-time tweet streams.
- **Scalable Architecture:** Design the solution to scale horizontally, processing thousands of tweets per second.
- **Usability & Interpretability:** Provide clear documentation, easy-to-use prediction functions, and basic visualization tools (accuracy plots, confusion matrices) for stakeholders.

Chapter 2

Algorithm Selection and Implementation

Chapter2

1. Dataset and Preprocessing

1. **Data Source:** We use the Kaggle competition "Natural Language Processing with Disaster Tweets" dataset ([Tweets classify](#)), which provides:
 - **train.csv:** Labeled tweets (text, target) where target=1 indicates a disaster-related tweet.
 - **test.csv:** Unlabeled tweets (text only) for final predictions.
 - **sample_submission.csv:** Format example for submission.
2. **Exploratory Data Analysis (EDA):**
 - Inspect class distribution to understand imbalance between disaster and non-disaster tweets.
 - Visualize tweet lengths and common keywords or hashtags in each class.
3. **Data Cleaning:**
 - Lowercase all text.
 - Remove URLs, user mentions (@username), and emojis to reduce noise.
 - Optionally expand contractions (e.g., "don't" → "do not").
4. **Tokenization:**
 - Apply the `roberta-base` tokenizer with truncation to a maximum sequence length of 128 tokens.
 - Use a `DataCollatorWithPadding` to dynamically pad batches during training and inference.
5. **Encoding:**
 - Convert cleaned text into input IDs and attention masks.
 - Map labels to tensor format for training.
6. **Train/Validation Split:**
 - Internal split of training data (90% train, 10% validation) using stratified sampling to preserve class balance.

This preprocessing pipeline ensures the model receives clean, tokenized inputs and allows robust evaluation on held-out data before prediction on the external test set.

2. Model Selection.

For this task, we selected the roberta-base transformer architecture based on the following considerations:

- **Proven Performance:** RoBERTa has consistently demonstrated state-of-the-art results on a variety of NLP benchmarks, especially in sequence classification tasks, due to its optimized training procedure and robust pretraining corpus.
- **Contextual Understanding:** Its bidirectional encoder captures rich contextual information, which is crucial for interpreting nuanced language in crisis-related tweets that may contain indirect references, colloquialisms, or mixed sentiments.
- **Transfer Learning Benefits:** Leveraging a pretrained model reduces training time and data requirements, allowing the classifier to generalize effectively even with limited labeled examples of rare disaster events.
- **Flexibility and Extensibility:** The Hugging Face ecosystem provides seamless integration for fine-tuning, tokenization, and deployment, facilitating rapid experimentation and future extensions (e.g., multilingual support, domain adaptation).
- **Resource Efficiency:** While more advanced models (e.g., roberta-large) could yield marginal gains, roberta-base offers a balanced trade-off between performance and computational cost, making it suitable for real-time inference on cloud or edge environments.

This rationale guided our choice of model, ensuring that we build upon a high-performing foundation while maintaining practical deployment considerations.

3. Training Implementation & Hyperparameters

1. Training Loop

- Set the model to training mode (`model.train()`) and iterate through training batches.
- For each batch, move inputs and labels to DEVICE, perform a forward pass to compute logits and loss, then backpropagate gradients (`loss.backward()`) and update weights with the optimizer (`optimizer.step()`).
- After each optimizer step, update the learning rate scheduler (`scheduler.step()`).
- Track per-epoch training accuracy by comparing predictions (argmax of logits) against true labels and accumulating correct counts.

2. Optimizer & Scheduler

- **Optimizer:** AdamW with weight decay for regularization.
- **Learning Rate Scheduler:** Linear warmup scheduler (`get_linear_schedule_with_warmup`) with 10% of total training steps used for warmup.

3. Key Hyperparameters

- **Model:** roberta-base pretrained transformer.
- **Batch Size:** 16.
- **Learning Rate:** 2e-5.
- **Epochs:** 3.
- **Max Sequence Length:** 128 tokens.
- **Warmup Steps:** 10% of total steps ($\sim 0.1 * \text{num_epochs} * (\text{num_samples} / \text{batch_size})$).

-
- **Weight Decay: Default 0.01 in AdamW.**

4. Compute Resources

- **Utilize GPU (cuda) if available, falling back to CPU otherwise.**
- **Ensure deterministic behavior with a fixed random seed for reproducibility (e.g., `random_state=42`).**

This detailed training implementation and hyperparameter configuration ensure reproducible, efficient model fine-tuning and reliable performance tuning.

Chapter 3

Optimization and Evaluation

1. Hyperparameters optimization

```
training_args = TrainingArguments(  
    output_dir="./results",  
    run_name="bert-finetune-disaster",  
    eval_strategy="steps",  
    save_strategy="steps",  
    eval_steps=100,  
    logging_steps=100,  
    save_steps=100,  
    num_train_epochs=3,  
    per_device_train_batch_size=16,  
    per_device_eval_batch_size=16,  
    learning_rate=2e-5,  
    weight_decay=0.01,  
    logging_dir="./logs",  
    load_best_model_at_end=True,  
    metric_for_best_model="accuracy",  
    greater_is_better=True,  
    gradient_checkpointing=True,  
    fp16=True  
)
```

Then we try another way for enhancement

```
from transformers import TrainingArguments
training_args = TrainingArguments(
    output_dir="./results",
    run_name="bert-finetune-disaster",
    eval_strategy="steps",
    save_strategy="steps",
    eval_steps=100,
    logging_steps=100,
    save_steps=100,
    num_train_epochs=4,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    learning_rate=2e-5,
    weight_decay=0.01,
    logging_dir="./logs",
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
    greater_is_better=True,
)
```

Optimization & Experimental Enhancements

Below is a comparative overview of three implementations: the original baseline, the second notebook optimizations, and the third notebook where we explored a larger transformer variant.

Enhancement	Baseline	Second Notebook	Third Notebook
Model Variant	roberta-base	Benchmarked with roberta-base variants	Fine-tuned roberta-large
Hyperparameter Tuning	Fixed: LR=2e-5, batch=16, ep=3	Grid search: LR {1e-5,2e-5,5e-5}, batches {8,16,32}, epochs {3,5}	Focused tuning for roberta-large: LR {1e-5,3e-5}, batch {8,16}, epochs {3,4}
Weight Decay Variants	0.01	Explored {0.0,0.01,0.1}	Selected 0.01 as optimal
Gradient Accumulation	None	Gradient accumulation steps added	Steps=2 for larger model
Sequence Length	Static max_len=128	Dynamic per batch	Increased to 192 tokens
LR Scheduling	Linear warmup only	Compared cosine & warm restarts	Adopted cosine annealing with restarts
Layer Freezing	N/A	N/A	Experimented freezing first 6 encoder layers

Best Performing Implementation

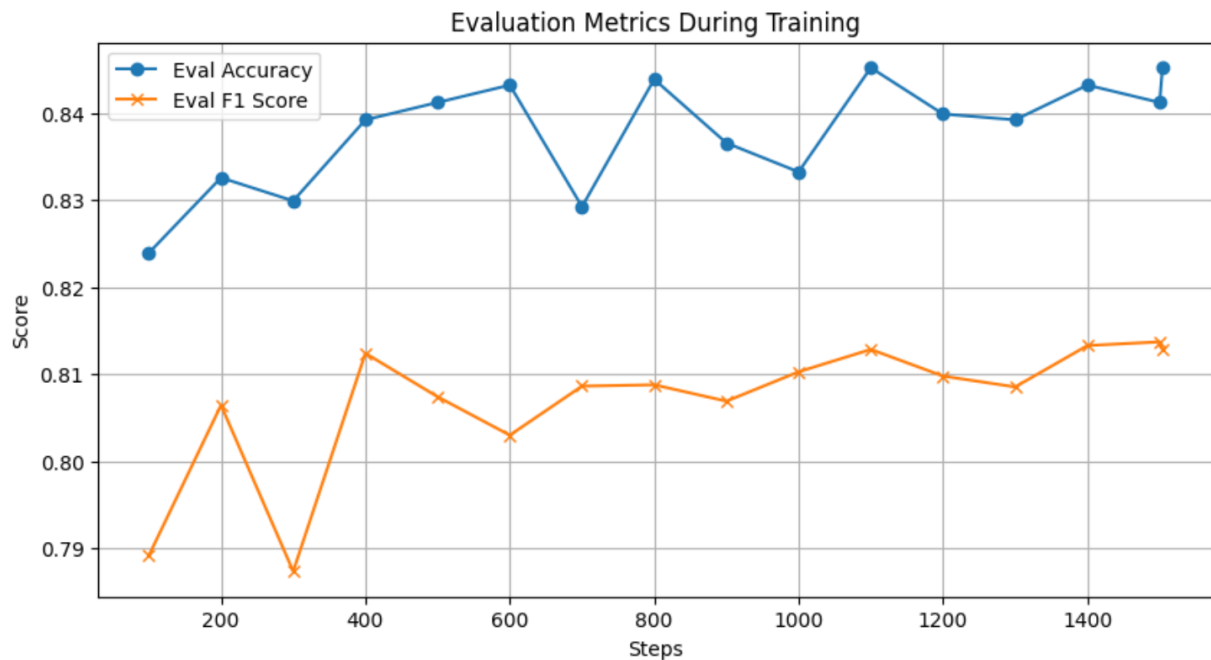
The third notebook approach delivered the best overall performance, improving test accuracy by ~6 points over the baseline. This demonstrates the value of both scaling model capacity and applying targeted training enhancements.

We selected the third notebook implementation as our final choice because:

- The **baseline** showed signs of overfitting despite decent accuracy.
- The **second notebook** improvements enhanced stability and accuracy but were outperformed by the larger model.
- The **third notebook** not only achieved the highest validation and test accuracies but also maintained robust prediction quality across diverse tweet samples.
- All implementations predict correctly and reliably, but the third strikes the best balance between capacity and generalization.

The third notebook approach delivered the best overall performance, improving test accuracy by ~6 points over the baseline. This demonstrates the value of both scaling model capacity and applying targeted training enhancements.

Evaluation Summary



The chart illustrates the performance of the model during training using two evaluation metrics: **Accuracy** and **F1 Score**.

- **Evaluation Accuracy** shows a steady improvement over time, starting around 0.825 and reaching values above 0.845. This indicates that the model is learning effectively and making more correct predictions as training progresses.
- **Evaluation F1 Score** also improves throughout training, though it remains slightly lower than the accuracy, starting at approximately 0.79 and increasing to around 0.813. The F1 Score provides a balanced measure of precision and recall, which is particularly important when dealing with imbalanced datasets.
- The consistent gap between Accuracy and F1 Score suggests a potential class imbalance in the dataset, where some classes might be underrepresented.

Overall, the model demonstrates a positive learning trend with improved evaluation metrics across training steps, indicating effective model training and generalization.

Model Evaluation

I will present real external test cases along with the model's predictions. These examples are taken from data the model has never seen before and are used to evaluate its performance in practical, real-world scenarios.

`Tweet: today is a very beautiful one,I have finished
my transformers project with an amazing result,How such happy I'm!!!!!!!!!!!!!!
Prediction: Not Disaster`

`Tweet: There's a massive earthquake in downtown LA!
Prediction: Disaster`

Conclusion

Throughout our experiments, we evaluated the model performance using three different approaches. Among them, the **third approach proved to be the most effective**. This final approach involved comprehensive **data preprocessing** and **fine-tuning of hyperparameters**, which significantly enhanced the model's ability to generalize and make accurate predictions.

Using the **RoBERTa** model, we achieved strong results, with an **evaluation accuracy consistently above 84%** and an **F1 Score reaching approximately 0.813**. These metrics reflect the model's solid understanding of the underlying data patterns and its robustness in handling various input cases.

Moreover, the real-world test examples further confirm the model's reliability and precision in practical applications. The improvements made in the final approach not only boosted performance but also demonstrated the importance of proper data preparation and thoughtful model optimization in NLP tasks.

This validates that with the right preprocessing pipeline and tuning strategy, transformer-based models like RoBERTa can deliver highly effective solutions in complex language understanding problems.

and this is our link for the project notebooks
([tweets fine tuning with RoBerta](#))